TideWAL: Timestamp-Integrated Dependency-Closed Parallel WAL for Main-Memory Transaction Processing

Shun Sasaki[1] Takayuki Tanabe[1] Hideyuki Kawashima[1]

概要: This paper proposes TideWAL, a dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL avoids conservative global-prefix durable acknowledgment while preserving recovery safety by acknowledging a transaction only after both its own log record and the durable frontier of its observed dependencies are persistent. In an ERMIA-style implementation, TideWAL reaches 460K durable acknowledgments per second on YCSB-B at 32 worker threads, 1.8× P-WAL throughput, while keeping pending durable commits to 59.

1. Introduction

1.1 Motivation

Transaction processing is a core abstraction behind data-intensive applications such as banking, payment processing, inventory management, ticket reservation, and online services that update shared state under high concurrency. A transaction processing system must provide both correct concurrent execution and crash recovery; in this sense, transaction processing consists of concurrency control (CC) and recovery [1]. A large body of work has studied scalable CC for multicore main-memory databases, including Silo, ERMIA, TicToc, SI, SSI, and SSN-style protocols [2], [3], [4], [5], [6], [7]. These protocols improve isolation and scalability by reducing centralized validation, timestamp, and version-management bottlenecks. In contrast, the recovery side of modern CC protocols has been explored less systematically: when a high-performance CC protocol is combined with crash durability, it is often unclear which recovery protocol preserves both safety and scalability.

Write-ahead logging (WAL) is the standard foundation for database recovery, with ARIES being the classical design [8], [9]. Modern systems have revisited WAL for multicore and fast-storage environments. SiloR parallelizes logging, checkpointing, and recovery for Silo; Passive Group Commit reduces synchronization overhead on non-volatile memory; and P-WAL partitions the log into multiple streams [10], [11], [12]. Although these systems differ in the target storage device, log format, and recovery procedure, they share an important lesson: writing log records in parallel is essential for making durability compatible with multicore transaction processing. At the same time, strong multiversion protocols such as SI, SSI, and SSN introduce timestamp and dependency structures that are not fully reflected in conventional parallel logging designs. The motivation of this work is therefore to clarify how P-WAL-style parallel logging should be combined with such timestamp-based CC protocols, especially SSN, without unnecessarily reintroducing global ordering and durable-prefix bottlenecks.

1.2 Problem

We target SSN because it is the most complex of the SI-family protocols we consider: it certifies serializability using dependency metadata, while the underlying engine still uses timestamped multiversion execution. As the baseline recovery protocol, we start from P-WAL, which is also used as a foundation of Shirakami, the transaction processing mechanism of Tsurugi [12], [13]. P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log

[1] Faculty of Environment and Information Studies, Keio University, Japan

streams. However, combining P-WAL with SSN exposes three problems.

First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

Second, workers can spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

Figure 1 summarizes these alternatives. Global-prefix acknowledgment is safe but may wait for unrelated slow shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before its dependencies are recoverable. The desired point is dependency-closed acknowledgment: acknowledge a transaction after its own log record and the transactions it depends on are durable.

In asynchronous durability protocols, logical commits and durable commit acknowledgments must be distinguished. A worker may continue after enqueueing a log record, but the transaction is not durable from the client's perspective until the system returns a durable acknowledgment. Therefore, this paper uses *durable-acknowledgment throughput* as the main throughput metric and reports it together with pending durable commits and tail latency.

1.3 Approach

This paper proposes *TideWAL*, a timestamp-integrated, dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL targets

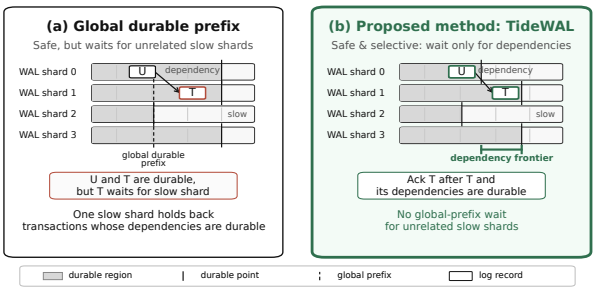

図 1 Durable acknowledgment policies in parallel WAL. Global-prefix acknowledgment is safe but can delay transactions whose dependencies are already durable because it waits for unrelated slow shards. Local-only acknowledgment avoids this delay but is unsafe. TideWAL uses dependency-closed acknowledgment, acknowledging a transaction only after its own log record and dependency frontier are durable.

timestamp-based engines such as ERMIA, where the transaction engine already computes a logical commit order. Its goal is not merely to maximize raw logical commit throughput, but to return durable commit acknowledgments safely without unnecessary global-prefix waiting.

TideWAL addresses the three problems above with three corresponding techniques.

A1. Commit timestamps as logical LSNs. TideWAL reuses the SSN/ERMIA commit timestamp as the logical LSN for recovery. WAL shards still maintain local physical positions, but the WAL layer does not allocate an additional global logical order. This removes duplicated global ordering between CC and recovery and eliminates WAL-side separate global LSN allocation on the commit path.

A2. Worker/flusher/committer separation. TideWAL decouples transaction execution from durable acknowledgment. Workers execute and validate transactions, assign commit timestamps, and enqueue log records. Flushers batch and persist shard-local logs. Committers observe durable-frontier advances and return durable acknowledgments. This pipeline keeps workers from directly blocking on `fdatasync` or on durable-prefix notification waits.

A3. Dependency-closed durable acknowledgment. TideWAL replaces global-prefix acknowledgment with dependency-closed acknowledgment. For each transaction, TideWAL maintains a dependency frontier that summarizes the log positions that must be recoverable before the transaction can be acknowledged. A transaction is acknowledged only when its own log record and this dependency frontier are durable. This preserves recovery

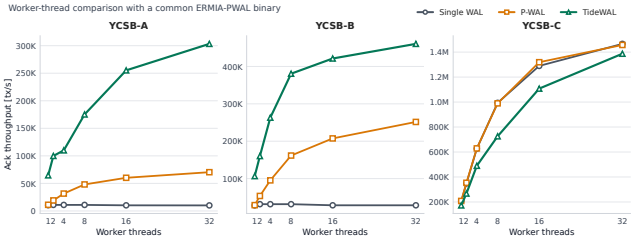

図 2 Durable-acknowledgment throughput by transaction worker threads. TideWAL improves update and mixed read-write workloads by combining parallel WAL logging, dependency-closed durable acknowledgment, and timestamp-integrated logical ordering.

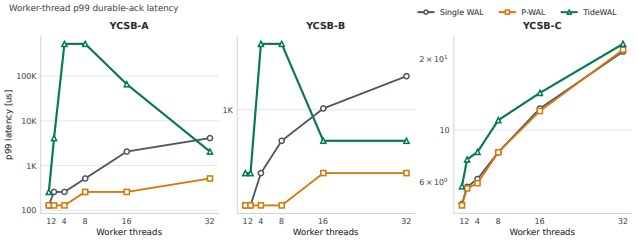

図 3 P99 durable-acknowledgment latency by transaction worker threads.

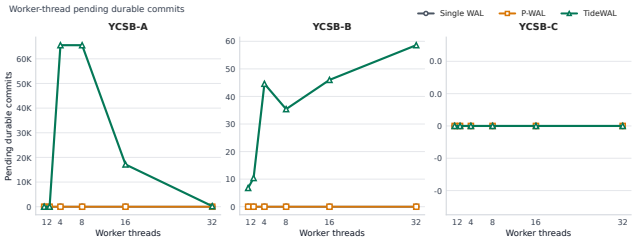

図 4 Pending durable commits by transaction worker threads.

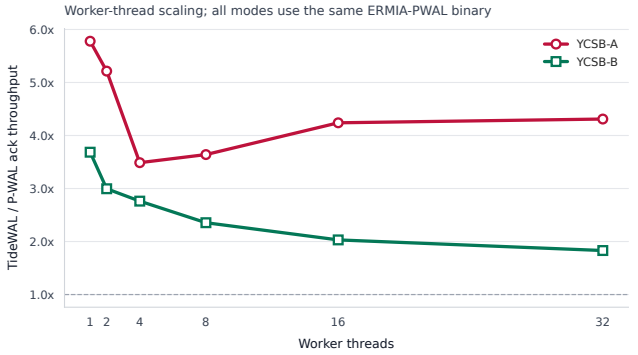

図 5 TideWAL throughput relative to P-WAL by transaction worker threads.

safety while avoiding waits for unrelated WAL shards.

We implement TideWAL in an ERMIA-style engine and evaluate it using YCSB workloads with the transaction worker count on the x-axis. The comparison uses the same ERMIA-PWAL binary for all WAL systems; Single WAL and P-WAL are selected by runtime WAL-mode flags, while TideWAL additionally runs background flusher/logger and committer threads. At 32 worker threads on YCSB-A, Single WAL achieves 10K durable acknowledgments per second and P-WAL achieves 69K, whereas TideWAL achieves 303K durable acknowledgments per second. On YCSB-B, Single WAL achieves 28K and P-WAL achieves 252K, whereas TideWAL achieves 460K durable acknowledgments per second. These results correspond to 4.3× and 1.8× improvements over P-WAL on YCSB-A and YCSB-B, respectively. TideWAL also keeps the number of pending durable commits small at 32 worker threads: 221 pending commits on YCSB-A and 59 pending commits on YCSB-B.

Figure 2 shows the worker-thread comparison. The results show that TideWAL improves update and mixed read-write workloads where durable logging remains on the critical path. We also evaluate YCSB-C, where all transactions are read-only and WAL records are skipped. In this case, durability work largely disappears; after using the same binary and a read-only empty-frontier fast path, Single WAL, P-WAL, and TideWAL perform similarly at 32 worker threads. We therefore use YCSB-C as a sanity check for read-only bookkeeping overhead rather than as evidence of WAL persistence performance.

TideWAL's timestamp integration is primarily a design simplification: it removes duplicated logical ordering between concurrency control and WAL durability by reusing ERMIA's commit timestamp as the logical recovery order. We do not rely on timestamp integration alone as a direct throughput optimization; the main performance effect comes from parallel logging, dependency-closed durable acknowledgment, and separating workers from durable-wait processing.

1.4 Organization

The rest of this paper is organized as follows. Section 2 reviews WAL, parallel WAL, timestamp-based concurrency control, and dependency requirements for recovery. Section 3 presents TideWAL's timestamp-integrated logical ordering, dependency-frontier construction, and worker/flusher/committer pipeline. Section 4 discusses the correctness of dependency-closed durable acknowledgment. Section 5 evaluates durable-acknowledgment throughput, pending durable commits, and tail latency. Section 6 discusses related work. Section 7 concludes the paper.

2. Background

3. Design

4. Correctness

5. Evaluation

5.1 Experimental Setup

We evaluate three systems: Single WAL, P-WAL, and TideWAL. We intentionally do not expose component-ablation modes in the main evaluation. Instead, we use internal counters and `perf` to explain why these three systems behave differently.

All systems use the same ERMIA-PWAL binary, `build/cc/ermia_pwal/ycsb_ermia_pwal.exe`. Single WAL and P-WAL are selected by runtime WAL-mode flags, and TideWAL uses the same binary with dependency-closed asynchronous durable acknowledgment enabled. Read-only WAL skipping is enabled for all systems. TideWAL uses group size 16, flush interval 50 μs, maximum pending durable commits 65536, and one committer thread. For worker counts 1, 2, 4, 8, 16, and 32, TideWAL uses 1, 1, 1, 2, 4, and 7 flusher/logger threads, respectively. The x-axis in the main figures is the number of transaction worker threads; TideWAL's background threads are reported separately in the tables.

Each YCSB run lasts 5 seconds and is repeated 5 times. We evaluate YCSB-A (50% read, 50% update), YCSB-B (95% read, 5% update), and YCSB-C (100% read). Throughput is durable-acknowledgment throughput, not logical commit throughput. For asynchronous logging, we also report pending durable commits and p99 durable-acknowledgment latency.

5.2 End-to-End Performance

Figure 2 shows durable-acknowledgment throughput. On update and mixed read-write workloads, TideWAL outperforms both Single WAL and P-WAL. At 32 worker threads, TideWAL reaches 303K acknowledgments per second on YCSB-A and 460K on YCSB-B, while P-WAL reaches 70K and 252K, respectively. These correspond to 4.3× and 1.8× speedups over P-WAL. Figure 5 reports the same comparison as TideWAL/P-WAL speedup.

Figure 3 and Figure 4 show the corresponding p99 latency and pending durable commits. TideWAL's gains are not produced by accumulating a large unbounded backlog: at 32 worker threads it has 221 pending commits on YCSB-A and 59 on YCSB-B. This is important because asynchronous durability can otherwise make throughput look high while delaying durable acknowledgments.

5.3 Internal Analysis

Table 2 explains the main performance difference using WAL counters. P-WAL removes the single shared log stream, but it still performs many small durable flushes. At 32 workers, P-WAL performs about 280K `fdatasync` calls on YCSB-A and 410K on YCSB-B, with only about 1.0 and 2.5 commits per `fdatasync`. TideWAL batches durable work: it performs about 85K and 134K `fdatasync` calls, with 14.3 and 13.8 commits per `fdatasync`. Thus, TideWAL reduces storage synchronization pressure and shifts the bottleneck away from frequent small flushes.

We also run `perf stat` and `perf record` for the three systems at 32 worker threads. Single WAL uses only about one CPU core on YCSB-A/B, which is consistent with a commit path serialized by durable logging. P-WAL uses more CPU cores, but `perf record` shows kernel synchronization as a dominant cost: `native_queued_spin_lock_slowpath` accounts for 39.6% of samples on YCSB-A and 31.8% on YCSB-B. TideWAL's top samples instead include `pthread_mutex_lock`, `TxExecutor::mergeVersionFrontier`, and `TxExecutor::publishReadFrontier`. This indicates that TideWAL reduces small-flush pressure, but its remaining cost comes from dependency-frontier metadata and queue management.

5.4 Read-Only Sanity Check

YCSB-C is read-only. With read-only WAL skipping enabled, it does not exercise WAL persistence: all three systems have zero WAL bytes and zero `fdatasync` calls. We therefore use YCSB-C only as a sanity check for read-only overhead. At 32 worker threads, Single WAL, P-WAL, and TideWAL reach 1.47M, 1.46M, and 1.39M transactions per second, respectively. TideWAL still performs read-side frontier bookkeeping: `perf record` shows `TxExecutor::mergeVersionFrontier` at 11.0% of samples on YCSB-C. Its committer does not perform durable work on this workload; the counters show zero waitlist registrations, zero pending commits, and zero `fdatasync` calls.

5.5 Timestamp Integration

TideWAL does not allocate a WAL-side global logical LSN. Instead, it reuses ERMIA's commit timestamp as the logical recovery order while retaining local physical

表 1 32-worker summary. Workers are transaction worker threads. TideWAL additionally uses 7 flusher threads and 1 committer thread at this point.

Workload	System	Workers	Streams	Flushers	Committers	Ack tps	p99 μs	Pending
YCSB-A	Single WAL	32	1	0	0	10082	4096	0
YCSB-A	P-WAL	32	32	0	0	70360	512	0
YCSB-A	TideWAL	32	7	7	1	303272	2048	221
YCSB-B	Single WAL	32	1	0	0	28489	2048	0
YCSB-B	P-WAL	32	32	0	0	251575	256	0
YCSB-B	TideWAL	32	7	7	1	460460	512	59
YCSB-C	Single WAL	32	1	0	0	1465872	22	0
YCSB-C	P-WAL	32	32	0	0	1457167	22	0
YCSB-C	TideWAL	32	7	7	1	1387033	23	0

表 2 I/O batching counters at 32 worker threads.

Workload	System	Ack tps	fdatasync	Commits/sync
YCSB-A	Single WAL	9759	39030	1.00
YCSB-A	P-WAL	69941	279538	1.00
YCSB-A	TideWAL	302880	84650	14.32
YCSB-B	Single WAL	27139	43670	2.49
YCSB-B	P-WAL	255003	409551	2.49
YCSB-B	TideWAL	463454	133959	13.84

表 3 Selected `perf stat` metrics at 32 worker threads.

Workload	System	CPU cores	Cycles/tx	Instr/tx	Ctx sw/sec
YCSB-A	Single WAL	1.20	306558	634424	127682
YCSB-A	P-WAL	18.36	662955	392300	1418583
YCSB-A	TideWAL	24.15	196890	141190	3290211
YCSB-B	Single WAL	1.08	99466	171821	156657
YCSB-B	P-WAL	16.45	162643	103787	1394703
YCSB-B	TideWAL	24.35	128797	77435	3880368

positions inside each WAL stream. Our counters confirm this design: for TideWAL, WAL-side global atomic allocation is 0 per transaction and WAL-side LSN allocation time is 0. This result should be read as evidence that TideWAL removes duplicated logical ordering between CC and durability, not as a standalone throughput ablation.

6. Related Work

7. Conclusion

謝辞

参考文献

[1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).

[2] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).

[3] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).

[4] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).

[5] Ports, D. R. K. and Grittner, K.: Serializable Snapshot Isolation in PostgreSQL, *Proc. VLDB Endow.*, Vol. 5, No. 12, pp. 1850–1861 (online), DOI: 10.14778/2367502.2367523 (2012).

[6] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Efficiently Making (Almost) Any Concurrency Control Mechanism Serializable, *The VLDB Journal*, Vol. 26, No. 4, pp. 537–562 (online), DOI: 10.1007/s00778-017-0463-8 (2017).

[7] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).

[8] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

[9] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).

[10] Zheng, W., Tu, S., Kohler, E. and Liskov, B.: Fast Databases with Fast Durability and Recovery Through Multicore Parallelism, *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '14, USENIX Association, pp. 465–477 (online), available from ⟨https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_wenting⟩ (2014).

[11] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Scalable Logging Through Emerging Non-

Volatile Memory, *Proc. VLDB Endow.*, Vol. 7, No. 10, pp. 865–876 (online), available from ⟨https://www.vldb.org/pvldb/vol7/p865-wang.pdf⟩ (2014).

[12] 孝明神谷，英之川島，喬　星野，修見建部，Komei, K., Hideyuki, K., Takashi, H., Osamu, T.：P-WAL: Paralell Write-ahead Logging, 情報処理学会論文誌データベース（TOD），Vol. 10, No. 1, pp. 24–39 (online), available from ⟨https://cir.nii.ac.jp/crid/1050282812885062144⟩ (2017).

[13] Kawashima, H.: Next-generation Database Tsurugi: Concurrency Control and Recovery Systems, *IPSJ Magazine*, Vol. 66, No. 4, pp. e9–e15 (online), DOI: 10.20729/0002001646 (2025).